Coinsult

Request your audit at coinsult.net

Advanced Manual
Smart Contract Audit

May 9, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

MedLeasePresale

0x86AdBaE80854D33CD53750324C46916664fF600b

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
Informational	3	0	2	1
Low-Risk	1	0	1	0
Medium-Risk	2	0	0	2
Critical-Risk	2	0	0	2

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
Informational	Does not compromise the functionality of the contract in any way.
Low-Risk	Won't cause any problems, but can be adjusted for improvement.
Medium-Risk	Will likely cause problems and it is recommended to adjust.
Critical-Risk	Will definitely cause problems, this needs to be adjusted.

Audit Summary

Project	MedLeasePresale
Website	-
Blockchain	BSC
Source Code	https://bscscan.com/address/0x86Ad-BaE80854D33CD53750324C46916664fF600b#code
Contract Address	0x86AdBaE80854D33CD53750324C46916664fF600b

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code. Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

Slither: Solidity static analysis frameworkRemix: IDE Developer ToolCWE: Common Weakness EnumerationSWC: Smart Contract Weakness Classification and Test CasesDEX: Testnet Blockchains

Table of Contents

Global Overview 2

Audit Summary 3

Audit Scope 4

Detailed Findings

Auto-success can lock refunds (DoS) 6

Refund can underflow after owner withdraw 7

Refund does not reduce global totals 8

Centralization: owner can end early 9

Rounding favors contract; user may overpay 10

Pause does not stop claim/refund/withdraw 11

No rescue for accidental token transfers 12

claim() can underflow depositedTokens 13

Disclaimer 14

Auto-success can lock refunds (DoS)

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-012: Resolved	buy() sets state=Successful when totalTokensOwed>=presaleTokensCap before collecting payment. If safeTransferFrom fails (no allowance/balance, token paused/blacklist), the tx reverts so state rolls back. But if it succeeds and later softCap is not met, the sale can never become Failed, blocking refunds and forcing Successful.

Code Snippet

```
bool willFinalize = totalTokensOwed >= presaleTokensCap;
if (willFinalize) { state = State.Successful; }
selectedToken.safeTransferFrom(msg.sender, address(this), actualPayment);
```

Recommendation

Do not set Successful solely on sold-out. Instead: (1) keep Active until endTime then finalize based on softCap, or (2) allow sold-out to end sale but still set final state based on totalRaised>=softCap, or (3) add an explicit 'SoldOut' state and allow refunds if softCap not met.

Note

On sold-out in buy(), state is set based on totalRaised >= softCap (Successful else Failed), preventing a sold-out sale from forcing Successful when softCap is not met.

Refund can underflow after owner withdraw

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-013: Resolved	If state becomes Failed, refund() subtracts amounts from totalRaised/totalRaisedUSDC/USDT. But withdrawFunds() can only happen in Successful; however state can be forced to Successful by sold-out logic. If later a bug/admin action sets Failed (or future upgrade), underflow would revert refunds. Current design tightly couples state correctness to arithmetic safety.

Code Snippet

```
totalRaised     -= (amountUSDC + amountUSDT);
totalRaisedUSDC -= amountUSDC;
totalRaisedUSDT -= amountUSDT;
```

Recommendation

Make state transitions impossible to contradict (see AUD-012). Additionally, consider tracking grossRaised separately from refundable balances, or guard with checks (e.g., require totals >= amounts) and rely on per-token balances for refunds.

Note

refund() adds explicit guards require(totalRaisedUSDC >= amountUSDC) and require(totalRaisedUSDT >= amountUSDT) before decrementing totals, preventing underflow; additionally AUD-012 fix reduces contradictory state risk.

Refund does not reduce global totals

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-004: Resolved	On Failed state, refund zeroes user contributions but does not decrement totalRaised/totalRaisedUSDC/totalRaisedUSDT. This leaves misleading totals and can break off-chain reporting and any future logic relying on these values.

Code Snippet

```
contributionsUSDC[msg.sender] = 0;
contributionsUSDT[msg.sender] = 0;
// totalRaised* not updated
```

Recommendation

Decrease totalRaised/totalRaisedUSDC/totalRaisedUSDT by refunded amounts (or clearly document totals as 'gross'). Prefer tracking both gross and net raised.

Note

refund() now decrements totalRaised, totalRaisedUSDC and totalRaisedUSDT by the refunded amounts before transferring USDC/USDT out.

Centralization: owner can end early

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-006: Resolved	Owner can finalize early once softCap is reached, ending the sale before endTime. This is a centralization risk: participants expecting more time may be excluded, and pricing tiers may be cut short.

Code Snippet

```
function finalizeEarly() external onlyOwner {
  require(totalRaised >= softCap);
  state = State.Successful;
}
```

Recommendation

Remove finalizeEarly or restrict it (e.g., only near endTime, or require a minimum duration/raised threshold). Clearly disclose this power to users.

Note

finalizeEarly() is not present; finalization is time-based via finalize() or sold-out auto-finalization in buy().

Rounding favors contract; user may overpay

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-008: Acknowledged	Token calculation uses integer division, rounding down tokens. Users may pay 'actualPayment' but receive slightly fewer tokens than expected due to rounding, especially across tiers. This is not a security bug but can cause user disputes.

Code Snippet

```
uint256 t = (use * tf) / price1; // integer division
```

Recommendation

Document rounding behavior. Optionally compute tokens with higher precision or refund dust (payment - actualPaymentUsed) to the buyer.

Note

calculate() still uses integer division and recomputes 'use' via division when capping by available tokens (e.g., use = (t * priceX) / tf), so rounding/dust can still favor the contract.

Pause does not stop claim/refund/withdraw

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-010: Acknowledged	Pausable is only applied to buy(). In emergencies, owner cannot pause claim/refund/withdraw flows. This may be intended, but it reduces incident response options if a bug is found in those functions.

Code Snippet

```
function buy(...) external ... whenNotPaused { ... }
function claim() external nonReentrant { ... }
```

Recommendation

Consider applying whenNotPaused to claim/refund/withdraw as well, or add separate emergency toggles. If intentional, document that pause only affects purchases.

Note

buy(), withdrawFunds(), withdrawUnsoldTokens() are pausable, but claim() and refund() remain callable while paused (explicitly intended per comments).

No rescue for accidental token transfers

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-011: Resolved	If users accidentally send unrelated ERC20s to the contract, there is no recovery function. This can strand tokens permanently.

Code Snippet

```
// no recoverERC20 / sweep function
```

Recommendation

Add an owner-only rescue function for arbitrary ERC20s excluding the sale token during Active, and excluding payment tokens until finalized/refunds complete.

Note

rescueERC20() added and blocks rescuing MEDL/USDC/USDT, allowing recovery of unrelated ERC20s.

claim() can underflow depositedTokens

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-014: Acknowledged	claim() decrements depositedTokens by the claimed amount, but depositedTokens is only increased on deposit() and is not reduced when owner withdraws unsold tokens. If owner withdraws too much (e.g., after sold-out, totalTokensOwed becomes 0), later claims can revert due to depositedTokens underflow, DoSing claims.

Code Snippet

```
function claim() external nonReentrant {
  ...
  totalTokensOwed -= amount;
  depositedTokens -= amount;
  token.safeTransfer(msg.sender, amount);
}

function withdrawUnsoldTokens() ... {
  uint256 balance = token.balanceOf(address(this));
  ... token.safeTransfer(owner(), withdrawable);
}
```

Recommendation

Do not mutate depositedTokens on claim(), or redefine it as 'contract MEDL balance' and update it on withdrawUnsoldTokens(). Prefer removing depositedTokens entirely and rely on token.balanceOf(address(this)) with strict invariants, or track 'availableForClaims' separately and decrease it on withdrawals.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.